

Projektdokumentation: WedO

Projekt: WedO - Wedding Photo Sharing App

App-Art: Native Mobile App für Android/iOS mit Expo React Native

Frameworks/Technologien: React Native, Expo, TypeScript, Firebase Auth, Firestore, Firebase Storage

Abgabe: Projektdokumentation als PDF, maximal 3 A4-Seiten

1. Vorstellung der App

WedO ist eine mobile App, mit der Hochzeitsgäste ihre Fotos unkompliziert mit dem Hochzeitsteam bzw. dem Fotografen teilen können. Der Fotograf erstellt in der App ein Event, teilt einen QR-Code oder einen Deep-Link mit den Gästen und verwaltet danach alle hochgeladenen Fotos in einer gemeinsamen Galerie.

Die App löst ein typisches Problem bei Hochzeiten: Fotos entstehen auf vielen privaten Smartphones, werden aber später oft nur teilweise oder über verschiedene Messenger geteilt. WedO bündelt diese Fotos pro Event an einem Ort. Gäste benötigen keinen Account, sondern scannen den QR-Code, bestätigen das Event und können sofort Fotos aufnehmen oder die Event-Galerie ansehen. Fotografen registrieren sich mit E-Mail und Passwort, erstellen Events und verwalten die hochgeladenen Inhalte.

2. Requirements und Umsetzung

CRUD-Operationen

Operation	Umsetzung in WedO
Create	Fotograf erstellt ein Hochzeits-Event; Gast oder Fotograf lädt ein Foto hoch.
Read	Events, Event-Details und Fotos werden aus Firestore gelesen und in Dashboard, Detailseite und Galerie angezeigt.
Update	Fotograf kann Event-Daten wie Titel, Datum und Location bearbeiten.
Delete	Fotograf kann Fotos aus der Galerie löschen; dabei werden Firestore-Dokument und Storage-Datei entfernt bzw. bereinigt.

Persistierende Daten

WedO nutzt Firebase als persistierende Datenbasis. Firestore speichert strukturierte Daten, Firebase Storage speichert die Bilddateien. Firebase Auth verwaltet die Fotografen-Accounts. Zusätzlich wird lokal per AsyncStorage das zuletzt gescannte Event gespeichert, damit Gäste beim nächsten App-Start schneller zurückkehren können.

Collection / Speicherort	Inhalt
weddings	Event-Titel, Ersteller, Share-Code, Datum, Location, Erstellzeitpunkt
photos	Event-ID, Bild-URL, Storage-Pfad, Upload-Rolle, Erstellzeitpunkt
Firebase Storage	Bilddateien unter weddings/{weddingId}/images/...jpg

Auswahlkriterium: Share-Dialog

Als drittes Kriterium wurde der Share-Dialog umgesetzt. Fotografen können die Einladung zum Event über den nativen Share-Dialog des Smartphones teilen. Auch einzelne Fotos können aus der Galerie heraus geteilt werden. Dadurch fühlt sich die App wie eine echte mobile Anwendung an und nutzt die nativen Funktionen des Betriebssystems.

3. Begründung der App-Art und des Frameworks

WedO wurde bewusst als Mobile App und nicht als Web-App umgesetzt, weil der zentrale Use Case direkt auf dem Smartphone stattfindet: Gäste fotografieren während einer Hochzeit, scannen QR-Codes, speichern Bilder

in der lokalen Galerie und teilen Inhalte über native Smartphone-Funktionen. Diese Anforderungen passen besonders gut zu einer nativen bzw. cross-platform mobilen App.

React Native mit Expo wurde gewählt, weil damit eine App für Android und iOS mit einer gemeinsamen Codebasis entwickelt werden kann. Expo vereinfacht typische Mobile-Funktionen wie Kamera, Media Library, Haptics, Clipboard und QR-Code-Flows. TypeScript sorgt für klarere Datenstrukturen und weniger Fehler bei Services, Navigation und Komponenten. Firebase passt gut zum MVP, weil Authentifizierung, Datenbank und Dateispeicher ohne eigenes Backend bereitgestellt werden.

4. Mockup und Funktionsweise

```

Start
|-- QR-Code scannen (Gast)
|   |-- Event bestätigen
|   |-- Foto aufnehmen
|   |-- Foto hochladen
|   |-- Event-Galerie ansehen
|
|-- Fotografen-Login
|   |-- Dashboard
|   |-- Event erstellen
|   |-- Event-Detailseite
|       |-- QR-Code / Link teilen
|       |-- Fotos ansehen
|       |-- Event bearbeiten
|       |-- Fotos speichern, teilen oder löschen

```

Startscreen: Der Nutzer entscheidet zwischen Gast-Flow und Fotografen-Login. Wenn ein Gast bereits ein Event gescannt hat, wird dieses Event erneut angeboten.

QR-Scanner: Gäste scannen den Event-Code oder geben einen Link manuell ein. Danach wird das Event geladen und bestätigt.

Gast-Kamera: Die App fragt Kamera- und Galerie-Berechtigungen an, nimmt Fotos auf, speichert sie lokal und lädt sie in Firebase hoch.

Dashboard: Fotografen sehen ihre Events, legen neue Hochzeiten an und öffnen die Detailansicht.

Event-Detailseite: Der Fotograf sieht QR-Code, Share-Link und Galerie. Fotos können geöffnet, gespeichert, geteilt oder gelöscht werden.

5. Learnings

Was lief gut? Die Kombination aus Expo und Firebase hat die Entwicklung eines funktionsfähigen MVPs stark beschleunigt. Besonders Kamera, Galerie, QR-Code, Authentifizierung und Cloud-Speicher ließen sich gut in einzelne Services und Screens aufteilen. Auch die Rollenlogik zwischen Gast und Fotograf war für die Demo gut nachvollziehbar.

Was lief schlecht? Einige native Funktionen benötigten genaue Berechtigungen und mussten auf echten Geräten getestet werden. Firebase-Regeln und Storage-Löschvorgänge waren fehleranfälliger als einfache Lese- und Schreiboperationen. Außerdem musste darauf geachtet werden, dass UI-Platzhalter nicht wie fertige Funktionen wirken.

Was würden wir beim naechsten Mal anders machen? Beim naechsten Projekt würden wir früher ein klares Datenmodell und Security-Konzept festlegen, Tests für die wichtigsten Services ergänzen und den Upload-Prozess robuster gestalten. Sinnvoll wären ausserdem Offline-Upload-Queue, Bulk-Download für Fotografen und eine Moderationsfunktion, bevor Fotos für alle sichtbar sind.

6. Fazit

WedO erfüllt die geforderten Kriterien: Die App besitzt CRUD-Funktionen, speichert Daten persistent in Firebase und nutzt den nativen Share-Dialog. Die App-Art und das Framework passen zum Anwendungsfall, da

Hochzeitsfotos direkt mobil aufgenommen, gespeichert und geteilt werden. Das Projekt zeigt einen realistischen MVP für eine Event-basierte Foto-Sharing-App.